Assignment No. 5

Title of the Experiment:
Setup your own cloud for Software as a Service (SaaS) over the existing LAN in your laboratory. Write your own code for cloud controller using open-source technologies to implement with HDFS. Implement the basic operations divide the file in segments/blocks and upload/download file on/from cloud in encrypted form.

Instructor's Name- Prof. S.N.Girme

Student's Name: Sukanya Gupta

Student's Roll no: 31416

Objectives:

- To set up a private cloud infrastructure using Microsoft Azure VMs connected over a Virtual Network (simulating a LAN). Deploy a distributed Hadoop HDFS cluster (1 Master + 2 Worker Nodes) that stores files by dividing them into segments/blocks replicated across multiple nodes. Build a custom Python Flask-based Cloud Controller (SaaS web application) that allows users to encrypt and upload files to the HDFS cluster, and download and decrypt them back implementing file segmentation, replication, encryption, and decryption entirely using open-source technologies.

Software & Resources Used:

- Cloud Provider: Microsoft Azure (Azure for Students subscription)
- Infrastructure: 3 Azure VMs Master-Node (20.235.81.30), Worker-Node-1 (20.219.7.28), Worker-Node-2 (20.219.189.117)
- Virtual Network: VNet (10.0.0.0/16, subnet 10.0.0.0/24) Resource Group: SaaS-Assignment-Group
- OS: Ubuntu Server 24.04 LTS on all nodes | VM Size: Standard_B2ats_v2
- Distributed Storage: Apache Hadoop 3.3.6 (HDFS NameNode on Master, DataNodes on Workers)
- Runtime: Java OpenJDK 11 (required by Hadoop)
- Cloud Controller / SaaS App: Python 3, Flask, cryptography (Fernet AES), hdfs (Python client)
- HDFS Replication Factor: 2 (each block stored on both Worker Nodes for fault tolerance)
- Port Configuration: SSH (22), HDFS NameNode Web UI (9870), SaaS Flask App (5000), Hadoop RPC (9000)

Theory / Background:

Software as a Service (SaaS) is a cloud computing model where a software application is hosted on the cloud and delivered to users over the internet users do not need to install or maintain anything locally. In this experiment, we built our own SaaS system from scratch using open-source components.

Hadoop HDFS (Hadoop Distributed File System) is the storage layer of the Apache Hadoop ecosystem. It is designed to store very large files by splitting them into fixed-size blocks (default 128 MB, configurable) and distributing those blocks across multiple machines called DataNodes. A NameNode acts as the master metadata server, tracking which blocks are stored on which DataNodes. This architecture provides parallel data access, fault tolerance through replication, and horizontal scalability.

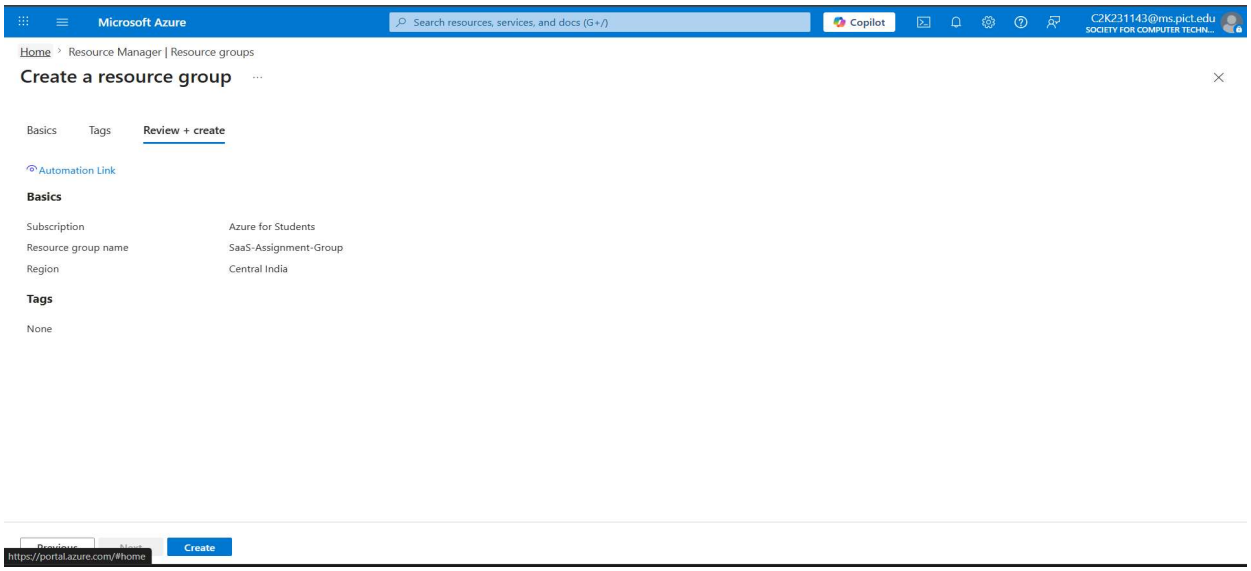
In this experiment, we configured a replication factor of 2, meaning every file block is stored on two different DataNodes simultaneously. If one worker node fails, the data is still available on the other this is automatic fault tolerance. The fsck (File System Check) tool was used to verify and visualize exactly which physical machines hold each block.

For the SaaS controller, we used Python Flask to build a web server running on port 5000 of the Master Node. The cryptography library's Fernet module provides symmetric AES-128 encryption. When a user uploads a file, the Flask app encrypts it using a secret key before writing it to HDFS. When a user downloads it, the app fetches the encrypted file from HDFS and decrypts it before sending it back ensuring data is never stored in plain text on the cloud.

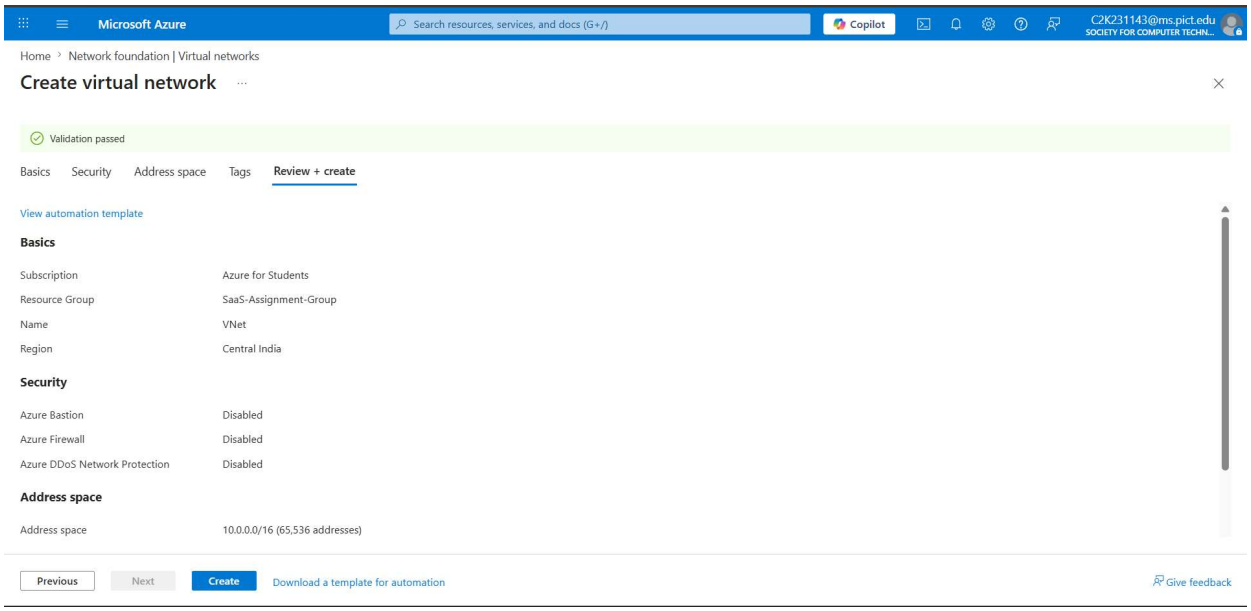
Steps / Procedure with Screenshots:

Phase 1: Building the Private Cloud LAN (Azure Infrastructure Setup)

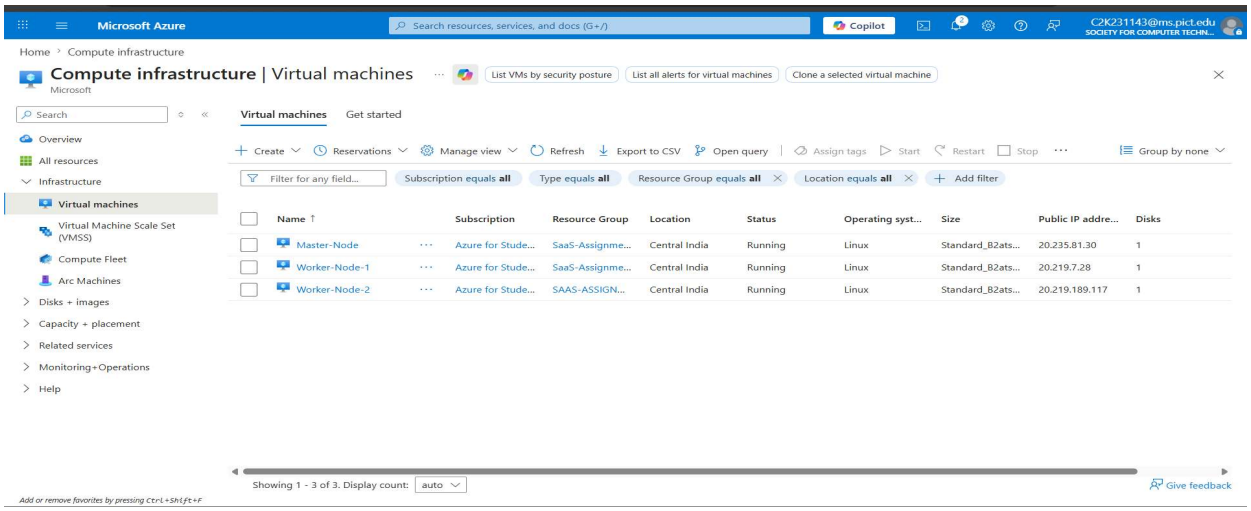
Step 1: We created a new Resource Group named 'SaaS-Assignment-Group' in the Azure portal under the 'Azure for Students' subscription, with region set to Central India. A Resource Group acts as a logical container that groups all related Azure resources VMs, networking, and storage together for easier management and cleanup.



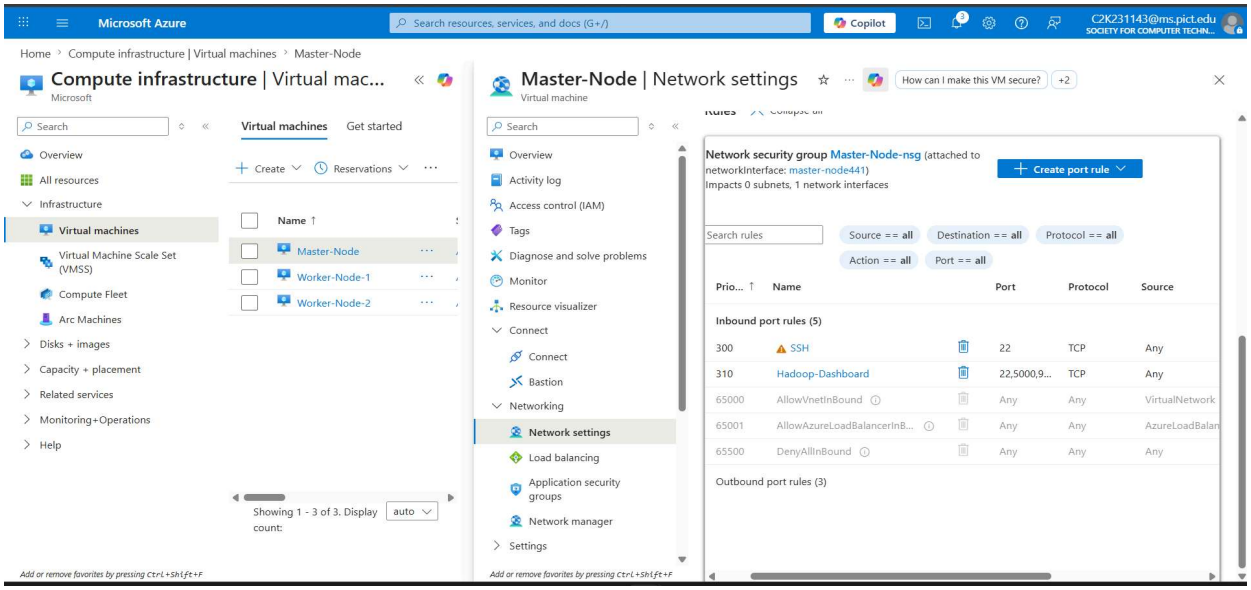
Step 2: We created a Virtual Network (VNet) named 'VNet' inside the SaaS-Assignment-Group, with address space 10.0.0.0/16 (65,536 available addresses) and subnet 10.0.0.0/24. This VNet acts as our private cloud LAN all three nodes (Master and Workers) were placed inside it so they can communicate with each other via private IP addresses without going over the public internet, exactly simulating a laboratory LAN.



Step 3: We provisioned three Ubuntu 24.04 LTS virtual machines inside the VNet Master-Node (Public IP: 20.235.81.30, Private IP: 10.0.0.4), Worker-Node-1 (20.219.7.28, Private IP: 10.0.0.5), and Worker-Node-2 (20.219.189.117, Private IP: 10.0.0.6) all in the SaaS-Assignment-Group, Central India region, Standard_B2ats_v2 size. All three VMs show status 'Running', confirming the cluster infrastructure is live.



Step 4: We configured the Master-Node's Network Security Group (NSG) with two inbound rules SSH (port 22) for remote terminal access, and 'Hadoop-Dashboard' (ports 5000, 9870+) to expose the HDFS NameNode Web UI and the Flask SaaS application to the internet. Worker Nodes only required SSH access since they communicate with the Master via the private VNet.



Phase 2: Configuring SSH Keys (Passwordless Inter-Node Communication)

Step 5: We set up SSH key-based authentication so the Master Node can communicate with both Worker Nodes without a password this is required for Hadoop's start-dfs.sh script to automatically start DataNodes on the workers. From Windows PowerShell, we uploaded the key to the Master Node, SSH'd in, moved the key to the secure SSH directory, and locked its permissions:

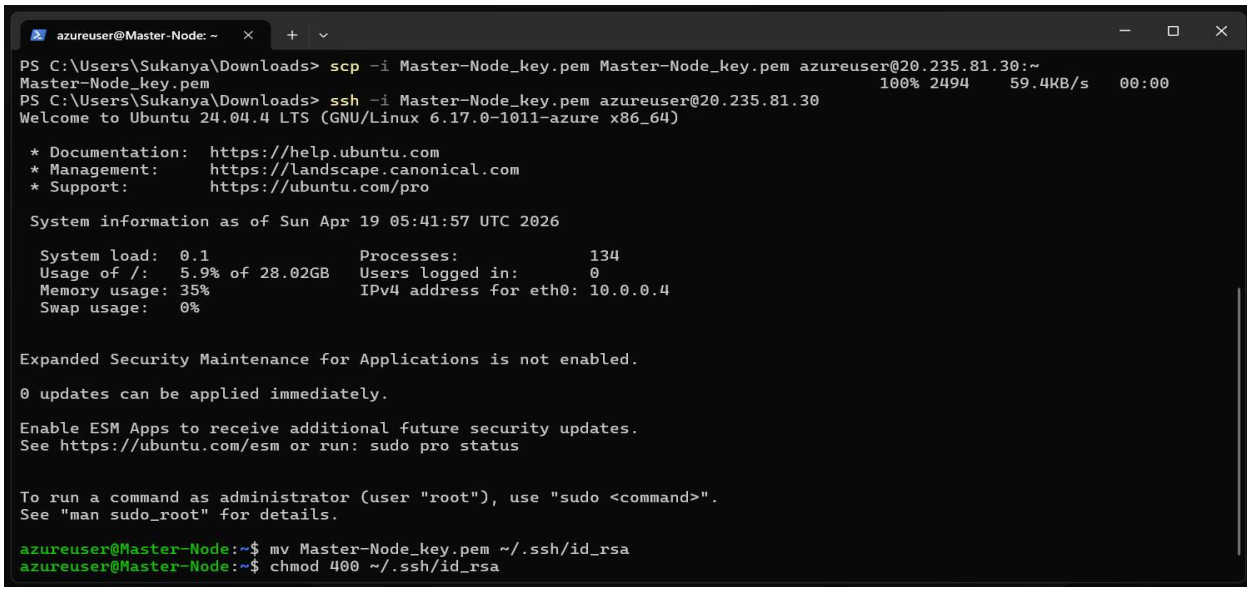
scp -i Master-Node_key.pem Master-Node_key.pem azureuser@20.235.81.30:~

ssh -i Master-Node_key.pem azureuser@20.235.81.30

mv Master-Node_key.pem ~/.ssh/id_rsa

chmod 400 ~/.ssh/id_rsa

The chmod 400 permission means only the file owner can read it SSH rejects keys that are too permissive. Because the key is now stored at the default location ~/.ssh/id_rsa, the Master Node can SSH into both Worker Nodes automatically without any password prompt.



Phase 3: Installing Java and Hadoop on All Nodes

Step 6: We updated the package lists and installed Java on every node. We used Java 11 specifically because newer versions (Java 21) caused compatibility issues with Hadoop 3.3.6 resulting in silent failures. Java is the engine that runs all Hadoop processes without it, Hadoop cannot start:

sudo apt update

sudo apt install openjdk-11-jdk -y

Step 7: On the Master Node, we downloaded Hadoop 3.3.6 from the Apache downloads server and extracted it:

wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz

tar -xzvf hadoop-3.3.6.tar.gz

We then configured four key files inside ~/hadoop-3.3.6/etc/hadoop/:

core-site.xml - set fs.defaultFS to hdfs://master-node.internal.cloudapp.net:9000 to define the HDFS cluster address.

hdfs-site.xml - set dfs.replication=2 so every block is stored on both Worker Nodes.

workers - listed the private IPs 10.0.0.5 and 10.0.0.6 so Hadoop knows which machines are workers.

hadoop-env.sh - set JAVA_HOME to the Java 11 installation path.

After configuring the Master, we used SCP to copy the entire configured Hadoop folder to both Worker Nodes so all nodes share identical configuration:

scp -r ~/hadoop-3.3.6 azureuser@10.0.0.5:~/

scp -r ~/hadoop-3.3.6 azureuser@10.0.0.6:~/

Phase 4: Formatting and Starting the HDFS Cluster

Step 8: We formatted the HDFS NameNode — a one-time operation that initialises the metadata filesystem on the Master Node, similar to formatting a new hard drive before first use:

~/hadoop-3.3.6/bin/hdfs namenode -format

Then we started the entire cluster with a single script. It used the SSH keys to automatically reach out to both Worker Nodes and start their DataNode processes remotely:

~/hadoop-3.3.6/sbin/start-dfs.sh

We verified the cluster was healthy by running jps (Java Process Status) on the Master it showed NameNode and on each Worker it showed DataNode confirming all nodes were active and connected:

jps

Phase 5: Building the SaaS Cloud Controller (Flask Web Application)

Step 9: On the Master Node, we created a Python virtual environment and installed the three required libraries:

python3 -m venv cloud_env

source cloud_env/bin/activate

pip install flask cryptography hdfs

Flask provides the web server and HTTP routing; cryptography provides Fernet symmetric AES-128 encryption; and hdfs provides a Python client to read and write files to HDFS. We created the HDFS storage directory and granted full permissions:

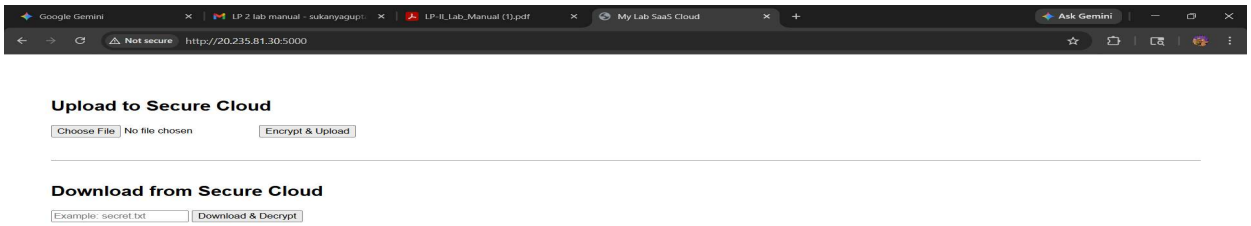
~/hadoop-3.3.6/bin/hdfs dfs -mkdir -p /user/cloud_storage

~/hadoop-3.3.6/bin/hdfs dfs -chmod 777 /user/cloud_storage

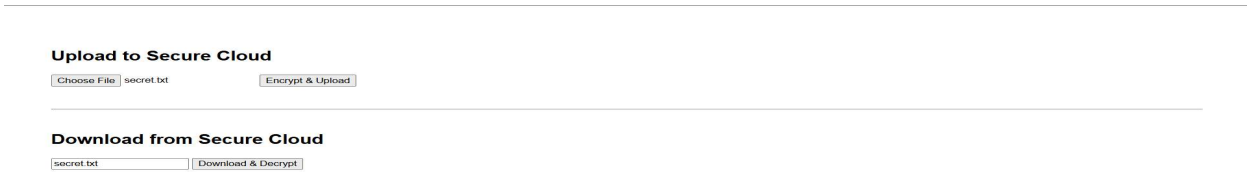
We then wrote the Flask application (app.py) implementing two HTTP endpoints: POST /upload (receives file, encrypts with Fernet, writes encrypted bytes to HDFS) and GET /download (reads encrypted file from HDFS, decrypts, sends plaintext back to browser). Finally, we started the server making the SaaS app live on port 5000:

python3 app.py

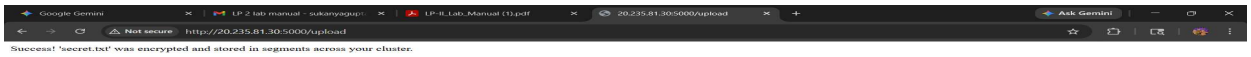
Step 10: The SaaS Cloud Controller web application was accessible in a browser at http://20.235.81.30:5000. The home page displayed two sections: 'Upload to Secure Cloud' with a file chooser and 'Encrypt & Upload' button, and 'Download from Secure Cloud' with a filename text box and 'Download & Decrypt' button. This is the complete user-facing interface of our custom-built SaaS system.



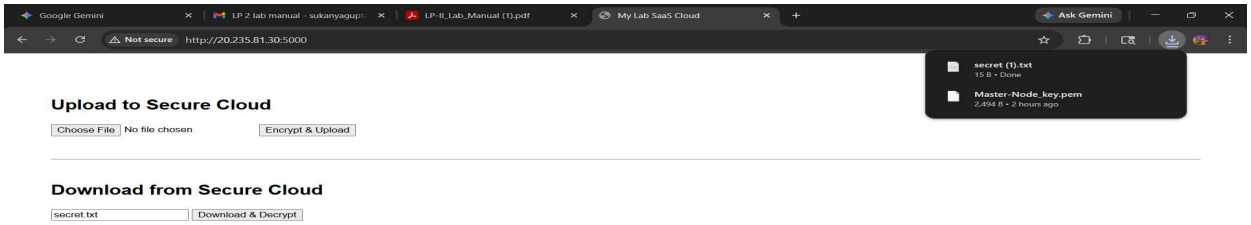
Step 11: We selected the file 'secret.txt' using the file chooser in the Upload section. The filename appeared next to the 'Choose File' button confirming the file was ready. Simultaneously, we entered 'secret.txt' in the Download filename box to prepare for testing the download flow.



Step 12: We clicked 'Encrypt & Upload'. The Flask application received the file, encrypted it using the Fernet symmetric key, and wrote the encrypted binary data to HDFS at /user/cloud_storage/secret.txt. The browser displayed the success confirmation message proving the file was encrypted and distributed across the HDFS worker nodes.



Step 13: We tested the Download & Decrypt feature by entering 'secret.txt' and clicking 'Download & Decrypt'. The Flask app fetched the encrypted file from HDFS, decrypted it using the Fernet key, and sent the original plaintext file to the browser. The Chrome download tray confirmed 'secret (1).txt' (15 B) was successfully downloaded proving the full encrypt-store-retrieve-decrypt cycle worked end-to-end.



Phase 6: Verification Proving File Segmentation and Replication (HDFS fsck)

Step 14 (KEY PROOF): We ran the HDFS File System Check command to perform an X-ray scan of the file stored in HDFS and print a detailed block location report:

~/hadoop-3.3.6/bin/hdfs fsck /user/cloud_storage/secret.txt -files -blocks -locations

The output confirmed: secret.txt (100 bytes) was divided into 1 block (blk_1073741825_1001) with replication factor 2. The block was physically stored on both Worker Node 1 (DatanodeInfoWithStorage[10.0.0.5:9866]) and Worker Node 2 (DatanodeInfoWithStorage[10.0.0.6:9866]). The cluster reported Status: HEALTHY, Number of data-nodes: 2, Default replication factor: 2, Missing blocks: 0, Missing replicas: 0 — definitively proving the distributed cloud storage is fully functional.

```
azureuser@Master-Node: ~

152.59.63.155 - - [19/Apr/2026 06:49:36] "GET /download?filename=secret.txt HTTP/1.1" 200 -
^C(cloud_env) azureuser@Master-Node:~~/hadoop-3.3.6/bin/hdfs fsck /user/cloud_storage/secret.txt -files -blocks -locations-ns
Connecting to namenode via http://master-node.internal.cloudapp.net:9870/fsck?ugi=azureuser&files=1&blocks=1&locations=1&path=%2Fuser%2Fcloud_storage%2Fsecr
et.txt
FSCK started by azureuser (auth:SIMPLE) from /10.0.0.4 for path /user/cloud_storage/secret.txt at Sun Apr 19 06:51:34 UTC 2026

/user/cloud_storage/secret.txt 100 bytes, replicated: replication=2, 1 block(s):  OK
0. BP-232236018-10.0.0.4-1776580131998:blk_1073741825_1001 len=100 Live_repl=2  [DatanodeInfoWithStorage[10.0.0.5:9866,DS-9581226d-8b01-4a54-8b69-09a2ebdd90
64,DISK], DatanodeInfoWithStorage[10.0.0.6:9866,DS-f5a60c48-dd16-40c1-b9ae-aad61f481e56,DISK]]


Status: HEALTHY
 Number of data-nodes:  2
 Number of racks:               1
 Total dirs:                    0
 Total symlinks:                0

Replicated Blocks:
 Total size:    100 B
 Total files:   1
 Total blocks (validated):      1 (avg. block size 100 B)
 Minimally replicated blocks:   1 (100.0 %)
 Over-replicated blocks:        0 (0.0 %)
 Under-replicated blocks:       0 (0.0 %)
 Mis-replicated blocks:         0 (0.0 %)
 Default replication factor:    2
 Average block replication:     2.0
 Missing blocks:                0
 Corrupt blocks:                0
 Missing replicas:              0 (0.0 %)
 Blocks queued for replication: 0

Erasure Coded Block Groups:
 Total size:    0 B
 Total files:   0
 Total block groups (validated):        0
 Minimally erasure-coded block groups:  0
 Over-erasure-coded block groups:       0
 Under-erasure-coded block groups:      0
 Unsatisfactory placement block groups: 0
 Average block group size:      0.0
```

Conclusion

In this experiment, we successfully designed and deployed a complete private cloud SaaS system from scratch using entirely open-source technologies. We built a private cloud LAN on Microsoft Azure by creating a Virtual Network (10.0.0.0/16) and provisioning three Ubuntu VMs one Master Node and two Worker Nodes simulating a real laboratory cluster. We installed Apache Hadoop 3.3.6 with Java OpenJDK 11 and configured HDFS with a replication factor of 2, meaning every file is automatically divided into blocks and stored on both Worker Nodes for fault tolerance. We wrote a custom Python Flask cloud controller implementing AES encryption (Fernet) for secure file upload and decryption on download, creating a complete SaaS interface accessible via a web browser. The HDFS fsck verification command provided definitive proof that the uploaded file (secret.txt) was segmented into blocks and replicated across both DataNodes (10.0.0.5 and 10.0.0.6), with the cluster reporting Status: HEALTHY and zero missing blocks or replicas. This experiment demonstrated all four key cloud computing concepts: a private cloud LAN (Azure VNet), distributed storage with file segmentation (HDFS blocks), fault-tolerant replication (factor 2), and a SaaS controller with encrypted upload/download (Flask + Fernet AES).